

07—其他材料

多智能体对比实验与系统测试报告

用可重复任务、运行轨迹与交付物证明系统能力

98.7%

Galen 硬门完成率

2.11

平均工具调用 / 次

9.2 s

中位端到端时延

本报告回答评委最关心的三个问题
Galen 是否比通用 Agent 更稳定地完成康复科研任务？专用架构的优势来自哪里？结果能否回到任务卡、原始运行与正式产物逐项核验？

主对比实验
15 张任务卡 × 5 次重复 × 2 套完整系统, 共 150 次对齐运行。

架构消融与回归
上下文压缩、无状态、执行契约、长任务看门狗和工程测试。

01 / 结论先行

不是 “能回答”，而是 “能按科研约束交付”

在本次冻结的康复科研任务集上，Galen 完成 74/75 次严格验收；Codex + DeepSeek 完成 21/75 次。

两组使用相同的 15 张任务卡、固定输入、重复编号与 DeepSeek V4 Flash 模型标签。差异主要出现在项目范围保持、数据治理、工具预算与正式产物交付，而不是文案流畅度。

观察维度	Galen	通用 CLI Agent 基线
硬门完成率	74/75, 98.7%	21/75, 28.0%
工具经济性	平均 2.11 次/运行	平均 6.12 次/运行
端到端时延	中位 9.2 秒	中位 19.3 秒
上下文修订	结构化项目状态吸收修订	易重新带回旧范围或旧数字
正式交付	同时检查事实、工具轨迹与可预览产物	常出现 “回答完成、产物未过硬门”

为何采用 “硬门” 而非主观打分

一次运行只有同时满足下列条件才计为完成：

$$\text{Pass} = \text{进程完成} \wedge \text{预算合规} \wedge \text{事实保留} \wedge \text{范围正确} \wedge \text{工具顺序正确} \wedge \text{产物可预览}.$$

这使得 “写得像一份报告” 和 “确实完成科研任务” 被清楚地区分开。每项失败都能回到具体断言与原始事件。

可直接引用的结论

在固定康复科研任务、固定模型标签和统一验收口径下，Galen 的状态化、契约化执行层显著提高了任务闭环完成率，同时降低了工具调用与完成时延。

02 / 实验设计

同任务、同输入、同指标，比较执行架构

任务组	PCTX 01-05	PDATA 01-05	PTOOL 01-05
考察能力	项目记忆、修订、范围切换、干扰抵抗	字段规范、缺失/重复、时间轴、来源冲突	检索、读写、失败恢复、正式交付
验收对象	当前事实与旧范围隔离	数据结构与质量记录	工具序列、预算、链接与产物
运行覆盖	5 卡 × 5 次	5 卡 × 5 次	5 卡 × 5 次

比较对象与覆盖

系统	任务卡	运行数	角色
Galen Full	15	75	完整状态层、数据契约与执行契约
Codex CLI + DeepSeek	15	75	完整重复的外部通用 Agent 基线
OpenCode + DeepSeek	15	15	全任务单次兼容性先导
OpenCode + DeepSeek	3	15	代表性任务 Repeat-5 稳定性观察

可重复性控制

冻结输入 同一任务文本、fixture、必需事实、禁用事实、允许工具和交付文件。	冻结判定 相同成功条件、工具预算、模型请求预算、超时与产物检查。
原始记录 每次运行保留 JSON 事件、工具轨迹、输出文件、断言结果与耗时。	统计方法 Wilson 95% 区间；以任务卡为重采样单位进行 20,000 次 bootstrap。

说明：OpenCode 为先导覆盖，未与 75 次重复结果合并；主比较为 Galen 与 Codex 两套完整重复系统。

Galen 在严格交付门槛下保持稳定完成

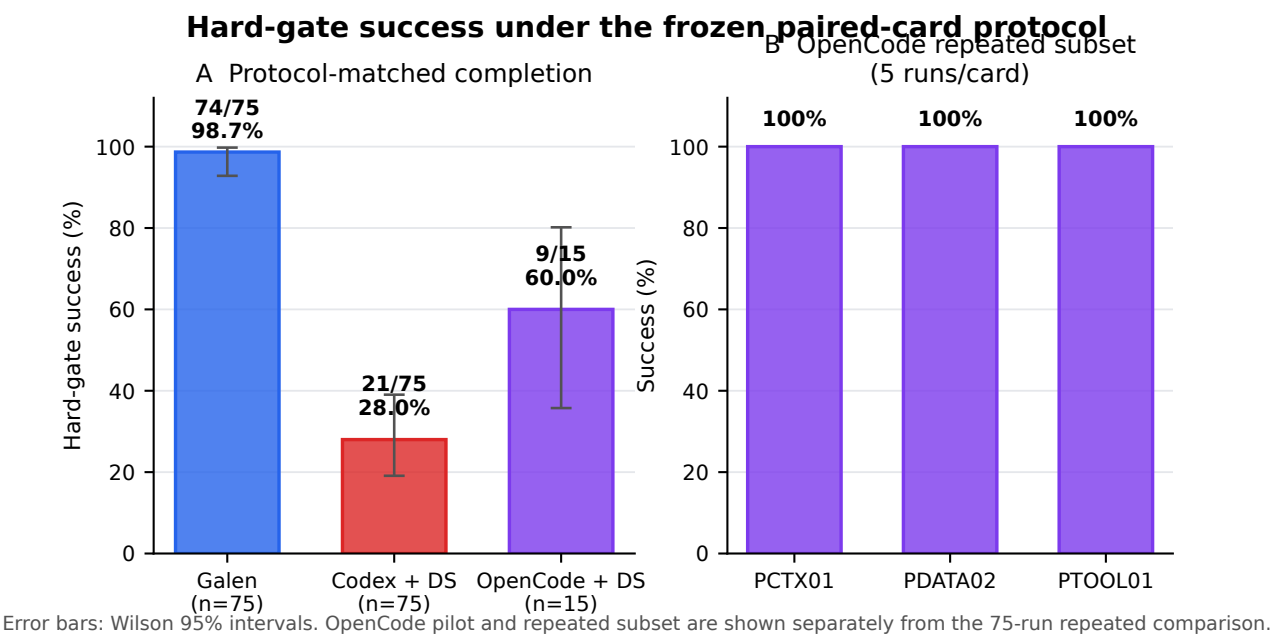


图 1: 冻结任务卡协议下的硬门完成率。误差线为 Wilson 95% 区间；OpenCode 的单次先导与重复子集分开展示。

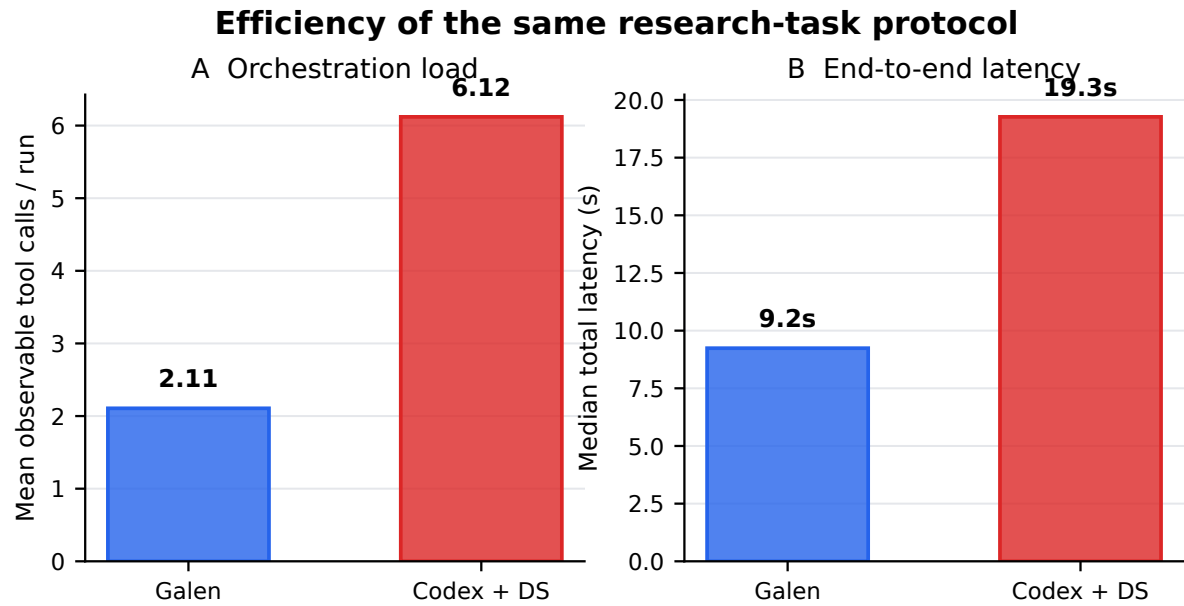
系统	通过/总数	完成率	Wilson 95% 区间
Galen	74/75	98.7%	92.8–99.8%
Codex + DeepSeek	21/75	28.0%	19.1–39.0%
OpenCode + DeepSeek (先导)	9/15	60.0%	35.7–80.2%

配对结果
相同 case/run index 下，Galen 独有通过 53 次，Codex 独有通过 0 次，共同通过 21 次，共同失败 1 次。按任务卡 bootstrap 的完成率差为 **70.7** 个百分点，95% 区间为 **52.0–88.0** 个百分点。

结果解释范围：该实验比较的是固定康复科研工作流中的执行可靠性，不是对基础模型通用智能的排名。

04 / 工具经济性

更少的工具动作，完成更完整的交付



Galen and Codex are both repeated 15-card × 5-run datasets. Tool calls are conservative event-normalized counts.

图 2: 相同 15 卡 × 5 次重复协议下的工具调用与端到端时延。

指标	Galen	Codex + DS	相对变化
平均可观察工具调用	2.11	6.12	减少 65.5%
中位端到端时延	9.2 s	19.3 s	缩短 52.3%
硬门完成率	98.7%	28.0%	提高 70.7 个百分点

工具经济性为何重要

科研 Agent 的调用成本不仅是 Token。无目的地遍历目录、重复检索、反复改写同一文件，会同时拉长时
间、增加失败面并削弱结果可复核性。Galen 通过项目状态、工具白名单、顺序约束和调用预算，把每次读
取、检索与写入绑定到明确的研究状态。

评审判断

Galen 的优势不是“少调用所以少做事”，而是在更高完成率的前提下，以更少工具动作交付了通过事实、
范围、轨迹和产物四重检查的结果。

05 / 失败边界

优势集中在哪里，薄弱点也清楚可见

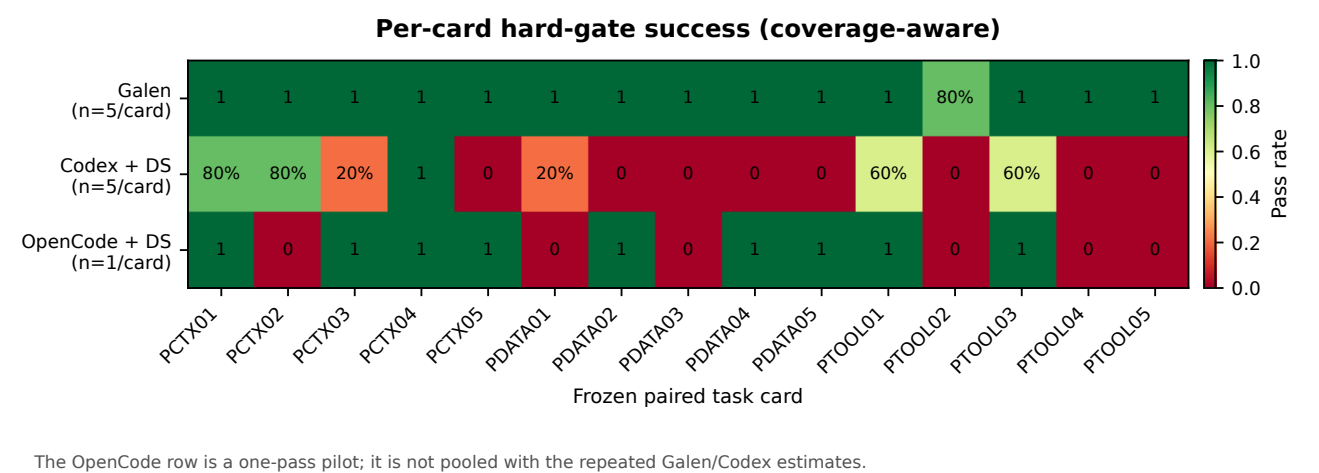


图 3: 逐任务卡硬门完成率。Galen 与 Codex 每卡 $n = 5$; OpenCode 为每卡 $n = 1$ 的兼容性先导。

能力区	Galen 表现	通用基线主要失败模式
上下文 PCTX	绝大多数任务 5/5	修订后重新带回旧病种、旧样本量或旧时间窗
数据治理 PDATA	五张任务卡均稳定完成	能解释规则，但未在预算内生成合规数据产物
工具与交付 PTOOL	仅 PTOOL02 出现 1 次预算越界	检索循环、工具顺序不符、失败恢复链中断

保留的真实失败

Galen 在 PTOOL02 有 1 次未通过：模型/工具请求超过任务设定的双预算。该记录未被重试覆盖，直接进入最终统计。它推动了长任务预算提示、细粒度进度与恢复点的后续改进。

06 / 架构消融

上下文能力来自状态层，而不是“模型多记一点”

配对上下文实验：完整历史 vs. 结构化上下文包

变体	通过	旧范围失败	平均输入 Token	TTFR 中位区间
完整历史 baseline	7/25	18	73,462	1.72–1.90 s
Galen 上下文包	25/25	0	15,601	1.30–1.59 s

结构化上下文包在本组实验中将输入 Token 平均值降低 78.8%，同时把完成率从 28% 提高到 100%。关键不是简单截断历史，而是显式保留研究问题、当前范围、排除方向、数据事实和已确认决策。

关键任务 Repeat-5 消融

变体	PCTX02	PCTX03	合计	主要现象
Full Galen	5/5	5/5	10/10	当前范围完整保留
Stateless	1/5	1/5	2/10	旧样本量、旧量表回流
Generic same runtime	0/5	1/5	1/10	上下文泄漏并触发预算超限
No execution contract	4/5	4/5	8/10	工具调用或事实约束偶发失守

消融结论
同一运行环境中关闭状态层后，两个高难度范围切换任务的通过数由 10/10 降至 2/10；这直接说明 Galen 的连续研究能力来自可执行项目状态，而不是聊天记录堆叠。

07 / 迭代与工程验收

测试不仅证明结果，也直接驱动版本修复

长任务可靠性回归

指标	修复前	修复后	变化
硬门通过	27/30	30/30	+3 次
Pass ⁵	66.7%	100.0%	+33.3 个百分点
Wilson 95% 下界	74.4%	88.6%	+14.2 个百分点
总耗时 P95	107.3 s	21.5 s	降低 80.0%

修复来源
Repeat-5 回归曾捕获两次读取文件后的长时间流式空转，以及一次真实观察 ID 未被带入产物。加入流空闲看门狗、可观察重试与病例 ID 契约后，6 张任务卡 × 5 次全部通过。

当前代码质量门

检查项	结果
Rust workspace 编译检查	通过
Galen 核心单元测试	219 passed / 0 failed / 1 ignored
前端 TypeScript 静态检查	通过
PDF 与产物可预览链路	通过

版本演进证据
严格对齐批次中，Galen 曾取得 72/75 (96.0%)；修复工具预算与交付链后，最终冻结批次达到 74/75 (98.7%)。报告同时保留两批记录，避免只展示最佳一次运行。

每一个数字都能回到原始记录

证据层	仓库位置
一键核验入口	07-其他材料/run_judge_benchmark.ps1
三张代表案例	07-其他材料/evidence-data/representative-cases/
冻结任务卡与 fixture	evals/cases/paired/
Galen 最终 75 次记录	evals/runs/large-full-galen-20260912-final.jsonl
Codex 最终 75 次记录	evals/runs/large-codex-deepseek-flash-20260912-complete.jsonl
最终统计摘要	evals/runs/large-comparison-20260912-final.md
上下文配对实验	evals/runs/paired-v1/20260911-formal-v1/
架构消融记录	evals/runs/architecture-v1/
长任务 Repeat-5 回归	evals/reports/ ais-t2-source-closed-watchdog-pro-repeat5-final-2026-08-25.md
统计图与 LaTeX 报告	output/pdf/galen-framework-comparison/

随本目录提供的评审核验材料

- run_judge_benchmark.ps1：无须 API Key，重算 150 次记录的完成率、工具调用与时延；
- evidence-data/：两份完整 JSONL、三张代表任务卡、双方轨迹、Galen 实际产物与 SHA-256 清单；
- 证据总表.md：从每项申报主张直接跳转到对应证据和核验动作；
- paired-benchmark/：15 张冻结任务卡、固定输入与自动判定条件；
- run_demo_checks.ps1：基础编译、单元测试与前端检查入口。

最终结论
Galen 已经形成 “固定任务—自动执行—硬门判定—失败定位—版本修复—重复验证” 的完整实验闭环。多智能体对比说明了专用科研执行架构的价值，消融实验说明了优势来源，原始记录与脚本保证结论可以复核。

[查看 Galen 公开仓库与提交分支](#)